순서가 있는 데이터: RNN과 LSTM

딥러닝 심화 1일차 · 1~5차시 · 책 11장 · 예제 b01~b26 개념 리뷰와 코드 해설

오늘의 시간표

차시	주제	책 범위	예제
1	순서 데이터의 모양, RNN 셀	11장 들어가며 ~ 11.2	b01 ~ b05
2	괄호 짝 맞추기, 기울기 소실과 폭주	11.3 ~ 11.4	괄호도구, b06 ~ b10
3	LSTM, GRU와 파라미터 비용, 긴 기억 시험	11.5 ~ 11.7	b11 ~ b14
4	층 쌓기·양방향, 꾸러미, 이산화탄소 예측	11.8 ~ 11.10	b15 ~ b22
5	속담 짓는 글자 LSTM 실습, 11장 정리	11.11 ~ 11.12	b23 ~ b26

차시마다 흐름

학습 목표 → 개념 → 코드와 실행 결과 → 직접 실행 문제(한 장) → 정답 → 확인 문제(한 장) → 정답 → 정리

실행 결과

슬라이드의 출력은 CPU로 직접 돌린 값
학습이 들어간 숫자는 교안과 조금 다를 수 있다

1 순서 데이터의 모양과 RNN 셀

책 11장 들어가며 ~ 11.2 · b01 ~ b05

학습 목표

1. 순서만 다른 데이터를 개수로는 구별할 수 없음을 확인한다
2. 창을 밀며 잘라 (배치, 길이, 특성) 모양을 만들고 읽는다
3. RNN 셀을 손으로 짜고 nn.RNN과 같은 값이 나오는지 맞춘다

책 범위

11장 들어가며 ~ 11.2

예제 파일

b01_order · b02_window · b03_cell
b04_nn_rnn · b05_length (모두 1초 안팎)

개념: 순서를 바꾸면 뜻이 바뀐다

"가격은 별로지만 맛은 최고"
"맛은 별로지만 가격은 최고"

글자 종류와 개수는 똑같다 → 칭찬과 불평이 정반대

오르는 주가 100 → 114
내리는 주가 114 → 100

평균·표준편차는 똑같다 → 추세는 정반대

그래서 필요한 것

개수·평균처럼 순서를 버리는 요약으로는 구별할 수 없다
→ 앞에서부터 차례로 읽으며 기억을 쌓는 모델 = 순환 신경망(RNN)

b01 순서를 버리면 구별이 안 된다

```
리뷰1 = "가격은 별로지만 맛은 최고"
리뷰2 = "맛은 별로지만 가격은 최고"
print(Counter(리뷰1) == Counter(리뷰2))      # 글자 개수만 비교

def 개수_벡터(문장):                          #글자 종류별 개수
    v = torch.zeros(len(글자들))
    for 글자 in 문장:
        v[번호[글자]] += 1
    return v
print(torch.equal(개수_벡터(리뷰1), 개수_벡터(리뷰2)))

오르는_중 = torch.tensor([100.0, 102.0, 105.0, 109.0, 114.0])
내리는_중 = 오르는_중.flip(0)                 #같은 숫자를 거꾸로
print(오르는_중.mean(), 내리는_중.mean())
print("마지막 날 - 첫날:", 오르는_중[-1] - 오르는_중[0],
      내리는_중[-1] - 내리는_중[0])
```

```
True
True
106.0 106.0
True
마지막 날 - 첫날: 14.0 -14.0
```

Counter

글자마다 몇 번 나왔는지 세는 사전
어느 글자가 먼저 왔는지는 버린다

flip(0)

0번 차원을 거꾸로 뒤집기
숫자 집합은 같고 순서만 반대

결론

다른 것은 순서뿐
마지막 날 − 첫날: +14 vs −14

11.1 b02 창을 밀며 잘라 내기

```
기온 = torch.tensor([3.0, 4.5, 6.0, 5.5, 7.0, 8.5, 8.0, 9.5, 11.0, 10.5])
창 = 3
조각들 = 기온.unfold(0, 창, 1)    # (차원, 창 크기, 한 번에 밀 칸)
print(조각들.shape)
X = 조각들[:-1]                    #마지막 창은 다음 날 정답이 없다
y = 기온[창:]                      #각 창의 바로 다음 날
for i in range(3):
    print(X[i].tolist(), "→", y[i].item())

X = X.unsqueeze(-1)                #하루에 숫자 1개 → 특성 1
print("입력:", tuple(X.shape), "정답:", tuple(y.shape))
print("같은 메모리를 쓰나?", 조각들.data_ptr() == 기온.data_ptr())
```

```
torch.Size([8, 3])
[3.0, 4.5, 6.0] → 5.5
[4.5, 6.0, 5.5] → 7.0
[6.0, 5.5, 7.0] → 8.5
입력: (7, 3, 1) 정답: (7,)
같은 메모리를 쓰나? True
```

unfold(0, 3, 1)

길이 3짜리 창을 한 칸씩 밀며 잘라 낸다
10개 → 창 8개 (10 − 3 + 1)

X와 y

X: 사흘치, y: 그다음 날
[3.0, 4.5, 6.0] → 5.5

unsqueeze(−1)

(7, 3) → (7, 3, 1)

복사가 아니다

unfold도 view처럼 같은 메모리를 다른 모양으로 읽는다

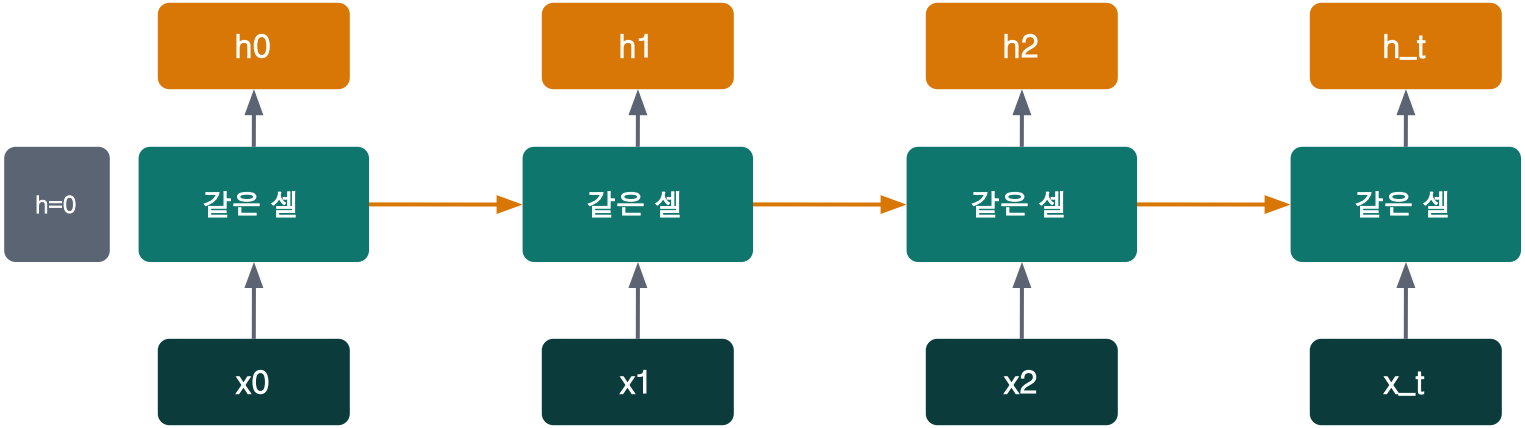
11.1 (배치, 길이, 특성)으로 읽기

기온만	기온·습도·바람	문장
(7, 3, 1)	(7, 3, 3)	(문장 수, 글자 수, …)
문제 7개, 각각 사흘 하루마다 숫자 1개	하루에 보는 숫자가 셋이면 특성이 3	마지막 자리는 글자 하나를 나타내는 숫자의 개수

자리마다 이름 붙여 읽기

순환 신경망의 입력은 언제나 3차원 (batch_first=True일 때)
첫 자리 = 배치(몇 개), 둘째 = 길이(몇 시점), 셋째 = 특성(한 시점에 숫자 몇 개)
이 버릇이 오늘 내내 쓰인다

11.2 개념: 메모장을 고쳐 쓰며 읽기



같은 셀(같은 가중치)을 시점마다 반복해서 쓴다

새 메모 h_t = tanh(x_t @ W_x + h_(t−1) @ W_h + b)

회의록 쓰는 사람: 발언 하나를 들을 때마다 요약을 다시 쓴다
요약 칸 크기는 정해져 있다 → 중요한 것은 남기고 덜 중요한 것은 밀어낸다
이 메모가 은닉 상태(hidden state) h, tanh가 −1~1로 묶는다

가중치는 몇 벌?

시점 수와 상관없이 W_x, W_h, b 한 벌 → 길이가 달라도 같은 모델

11.2 b03 손으로 만드는 RNN 셀

```
입력_크기, 메모_크기 = 3, 4
W_x = torch.randn(입력_크기, 메모_크기) * 0.5    # 읽은 것 → 메모
W_h = torch.randn(메모_크기, 메모_크기) * 0.5    # 지난 메모 → 새 메모
b = torch.zeros(메모_크기)

def rnn_셀(x, h):
    return torch.tanh(x @ W_x + h @ W_h + b)

순서열 = torch.randn(2, 5, 입력_크기)    # (배치 2, 길이 5, 특성 3)
h = torch.zeros(2, 메모_크기)             # 처음엔 빈 메모장
메모들 = []
for t in range(순서열.shape[1]):          # 한 시점씩 차례로
    h = rnn_셀(순서열[:, t], h)           # (2, 3), (2, 4) → (2, 4)
    메모들.append(h)
출력 = torch.stack(메모들, dim=1)         # (2, 5, 4)
```

```
t=0: 첫 문장의 메모 [0.034, 0.412, -0.082, -0.311]
t=1: 첫 문장의 메모 [0.419, 0.388, -0.072, 0.271]
t=2: 첫 문장의 메모 [0.203, -0.596, -0.832, 0.29]
t=3: 첫 문장의 메모 [0.395, -0.032, -0.8, -0.651]
t=4: 첫 문장의 메모 [-0.498, -0.029, 0.382, 0.437]
모든 시점의 메모: (2, 5, 4) 마지막 메모: (2, 4)
```

순서열[:, t]

모든 문장의 t번째 시점만 → (2, 3)

for가 핵심

앞 시점의 h가 다음 시점의 입력으로 들어간다
그래서 시점을 동시에 계산할 수 없다

출력과 마지막 메모

출력: 모든 시점의 메모 (2, 5, 4)
h: 마지막 메모 = 전체 요약 (2, 4)

11.2 b04 nn.RNN: 같은 일을 하는 부품

```
rnn = nn.RNN(input_size=3, hidden_size=4, batch_first=True)
for 이름, p in rnn.named_parameters():
    print(f"{이름:14} {tuple(p.shape)}")

# 같은 가중치로 손으로 계산 (PyTorch는 x @ W.T 꼴로 보관)
h = torch.zeros(2, 4)
for t in range(5):
    h = torch.tanh(순서열[:, t] @ rnn.weight_ih_l0.T + rnn.bias_ih_l0
                   + h @ rnn.weight_hh_l0.T + rnn.bias_hh_l0)

출력, h_n = rnn(순서열)
print((h - h_n[0]).abs().max())
print(torch.allclose(출력[:, -1], h_n[0]))
```

```
weight_ih_l0   (4, 3)
weight_hh_l0   (4, 4)
bias_ih_l0     (4,)
bias_hh_l0     (4,)
출력: (2, 5, 4) h_n: (1, 2, 4)
손 계산과 차이: 5.960464477539063e-08
출력의 마지막 시점 == h_n? True
```

이름 읽기

ih: input→hidden (W_x)
hh: hidden→hidden (W_h)
l0: 0번 층

차이 6e-8

교안은 0.0, 여기선 6e-8
계산 순서에 따른 float 반올림 차이일 뿐
같은 계산

가중치 모양이 (4, 3)

PyTorch는 (출력, 입력)으로 보관해서 .T로 곱한다

11.2 nn.RNN이 돌려주는 두 가지

출력 (2, 5, 4)

모든 시점의 메모

시점마다 무언가를 내야 하는 문제
예: 다음 글자 맞히기 (5차시)

h_n (1, 2, 4)

마지막 시점의 메모

다 읽고 한 번 판단하는 문제
예: 괄호 짝 판정 (2차시)

h_n 맨 앞의 1

층 수 (4차시에는 층 수 × 방향 수)
층이 하나면 출력[:, −1]과 h_n[0]은 같은 값

11.2 b05 길이가 달라도 같은 모델

```
rnn = nn.RNN(input_size=3, hidden_size=4, batch_first=True)
print("파라미터 수:", sum(p.numel() for p in rnn.parameters()))
for 길이 in [1, 5, 500]:                          #같은 모델에 길이만 다르게
    출력, h_n = rnn(torch.randn(2,길이, 3))

h0 = torch.zeros(1, 2, 4)                          # 층 수, 배치, 메모 크기)
앞부분, h_중간 = rnn(torch.randn(2, 3, 3), h0)
뒷부분, h_끝 = rnn(torch.randn(2, 2, 3), h중간)    # 이어서 읽기

rnn_기본 = nn.RNN(input_size=3, hidden_size=4)   # batch_first없음
출력, _ = rnn_기본(torch.randn(5, 2, 3))         # 길이, 배치, 특성)
```

```
파라미터 수: 36
길이   1: 출력 (2, 1, 4), h_n (1, 2, 4)
길이   5: 출력 (2, 5, 4), h_n (1, 2, 4)
길이 500: 출력 (2, 500, 4), h_n (1, 2, 4)
이어 읽은 출력: (2, 3, 4) (2, 2, 4)
batch_first=False 출력: (5, 2, 4)
```

파라미터 36개

W_x 12 + W_h 16 + 편향 8
길이 1이든 500이든 그대로

이어 읽기

앞부분의 마지막 메모를 다음 호출의 h0로 넘기면 끊김 없이 이어진다 (5차시 속담 짓기에서 씀)

batch_first 주의

기본값은 (길이, 배치, 특성)
이 책은 언제나 batch_first=True

직접 실행: 1차시

복사본에서 고치고, 먼저 예상해서 적기

1. b04를 실행해 손 계산과 nn.RNN의 차이를 확인하라. 0인가?

2. b02 복사본에서 창 = 3을 5로 바꾸면 조각 수와 입력 X 모양은?

3. b05에서 hidden_size=4를 8로 (h0도 (1, 2, 8)로) 바꾸면 파라미터 수는? 손으로 먼저

   `W_x + W_h + 편향 2개`

직접 실행 정답: 1차시

1

여기서는 5.96e-08 (교안은 0.0)
float32 반올림 수준의 차이 → 같은 계산

2

조각들 (6, 5): 10 − 5 + 1 = 6개
X (5, 5, 1), y (5,): 마지막 창은 정답이 없어 5개

3

104개
8×3 + 8×8 + 8 + 8 = 24 + 64 + 16

확인 문제: 1차시

1 nn.RNN(3, 4, batch_first=True)에 길이 500짜리 입력 2개를 넣으면 h_n의 모양은?

가 (2, 500, 4)　나 (1, 2, 4)
다 (500, 2, 4)　라 길이가 너무 길어 오류

2 nn.RNN(input_size=10, hidden_size=20)의 파라미터 수는?

3 batch_first=False인 RNN(3, 4)에 (2, 5, 3)을 넣으면 출력과 h_n의 모양은?

4 기온이 100일치면 창 3으로 만든 문제(X)는 몇 개?

확인 문제 정답: 1차시

1 나 (1, 2, 4)
(층 수, 배치, 메모 크기). 길이와 상관없다

2 640개
20×10 + 20×20 + 20 + 20

3 출력 (2, 5, 4), h_n (1, 5, 4)
길이 2, 배치 5로 읽어 버린다. 오류가 안 나서 더 위험

4 97개
창 98개(100 − 3 + 1), 마지막 창은 정답이 없어 뺀다

1차시 정리

01

순서만 다른 데이터는 개수로 구별할 수 없다. 순서 데이터는 (배치, 길이, 특성), 창을 밀며 잘라 만든다

02

RNN: h_t = tanh(x_t @ W_x + h_(t−1) @ W_h + b). 모든 시점이 같은 가중치라 어떤 길이든 받는다

03

출력은 모든 시점의 메모, h_n은 마지막 메모. batch_first=True를 잊지 않는다

다음: 순서를 알아야만 풀리는 문제, 그리고 RNN의 약점

2

괄호 짝 맞추기, 기울기 소실과 폭주

책 11.3 ~ 11.4 · 괄호도구, b06 ~ b10

학습 목표

1. 순서가 필요한 문제에서 MLP와 RNN의 차이를 숫자로 본다
2. 길이를 받아 주는 것과 처음 보는 길이에서 잘하는 것을 가른다
3. 시점별 기울기를 재어 소실과 폭주를 확인하고 처방을 고른다

책 범위

11.3 ~ 11.4

예제 파일

괄호도구 · b06_paren_data · b07_paren_models (4초)
b08_paren_length (2초) · b09_power · b10_grad_time

11.3 괄호도구.py: 짝 판정과 데이터 만들기

```
def 짝맞나(괄호):
    깊이 = 0
    for 글자 in 괄호:
        깊이 += 1 if 글자 == "(" else -1
        if 깊이 < 0:            #  닫는 게 먼저 많아지면 실패
            return False
    return 깊이 == 0            #  끝에서 0이면 성공

def 괄호_만들기(개수, 길이들, 씨앗):
    # 짝 맞는 괄호를 만들고, 홀수 번째는 두 글자를 맞바꿔 짝을 깬다
    # → 짝 맞음/틀림 반반, 두 쪽 모두 '('와 ')' 개수는 같다

def 원핫으로(괄호들):
    # '(' → 1번, ')' → 2번, 빈칸(패딩) → 0번 칸이 1인 원-핫
    채운 = pad_sequence(번호들, batch_first=True, padding_value=0)
    return one_hot(채운, 3).float(), 길이
```

짝맞나

앞에서부터 열린 괄호의 깊이를 센다
0 아래로 내려가는 순간이 있거나 끝이 0이 아니면 틀림

"(()())" vs "())(()"

둘 다 여는 괄호 셋, 닫는 괄호 셋
앞은 맞고 뒤는 틀림 → 순서가 결정

원-핫 3칸

[빈칸, (,)]
'(' = [0, 1, 0], ')' = [0, 0, 1]

11.3 b06 개수로는 절대 못 가르는 데이터

```
print(짝맞나("(()())"), 짝맞나("())(()"), 짝맞나("(()"))
괄호들, 정답 = 괄호_만들기(3000, 길이들=[20], 씨앗=0)
for 괄호, 답 in zip(괄호들[:4], 정답[:4].tolist()):
    print(괄호, "짝 맞음" if 답 else "짝 틀림")
print("짝 맞음 비율:", 정답.float().mean().item())
print("'('와 ')' 개수가 늘 같은가:",
      all(괄호.count("(") == 괄호.count(")") for 괄호 in 괄호들))
X, 길이 = 원핫으로(괄호들[:2])
print("원-핫 모양:", tuple(X.shape))
```

```
True False False
(((()(((())(()))))())) 짝 맞음
()()()()()())((()()) 짝 틀림
()((((()))(())()(())) 짝 맞음
()())(()(()()))(()() 짝 틀림
짝 맞음 비율: 0.5
'('와 ')' 개수가 늘 같은가: True
원-핫 모양: (2, 20, 3)
```

두 글자 맞바꾸기

틀린 괄호는 맞는 괄호에서 두 글자를 바꿔 만들었다 → 여는 10개, 닫는 10개로 늘 같다

반반

찍어도 50%

(2, 20, 3)

(배치, 길이, 특성)
특성 3 = 원-핫 칸 수

11.3 b07 세 모델의 대결

```
class 개수_모델(nn.Module):              #글자 개수만 본다
    def forward(self, x):
        return self.mlp(x.sum(dim=1))     # 배치, 20, 3) → (배치, 3)

펼친_MLP = nn.Sequential(nn.Flatten(), nn.Linear(20 * 3, 64),
                         nn.ReLU(), nn.Linear(64, 2))    # 괄을 한 줄로

class 순환_모델(nn.Module):              #한 글자씩 읽는다
    def __init__(self, 메모_크기=32):
        super().__init__()
        self.rnn = nn.RNN(3, 메모_크기, batch_first=True)
        self.판단 = nn.Linear(메모_크기, 2)
    def forward(self, x):
        _, h_n = self.rnn(x)
        return self.판단(h_n[-1])         #마지막 메모로 판단
# 같은 학습: Adam lr=3e-3, 배치 64, 30에폭
```

```
개수만 보는 MLP: 파라미터 194개, 학습 50.0%, 검증 50.0%
펼쳐서 보는 MLP: 파라미터 4,034개, 학습 100.0%, 검증 99.7%
RNN: 파라미터 1,250개, 학습 100.0%, 검증 100.0%
```

개수 MLP 50%

x.sum(dim=1)로 길이 방향을 더하면
순서가 사라진다 → 원리적으로 못 푼다

펼친 MLP 4,034개

1번 자리, 2번 자리 …를 위한 가중치를
따로따로 배운다
첫 층은 60칸 입력만 받는다

RNN 1,250개

모든 자리에 같은 가중치를 돌려 쓴다
질문: 길이 24, 30, 40을 주면?

11.3 b08 처음 보는 길이

```
# 앞 예제처럼 길이 20 괄호로 RNN 학습 (30에폭)
for 길이 in [20, 24, 30, 40]:
    괄호들, 정답 = 괄호_만들기(1000, [길이], 씨앗=2)
    X, _ = 원핫으로(괄호들)
    정확도 = (모델(X).argmax(dim=1) == 정답).float().mean().item()
    try:
        펼친_MLP(X)                          # 6만 입력만 받는 MLP
    except RuntimeError as e:
        ...                                  #모양 오류를 잡아 출력
```

```
길이 20: RNN 정확도 100.0% / 펼친 MLP 실행됨
길이 24: RNN 정확도 87.9% / 펼친 MLP 모양 오류 (1000x72 and 60x64)
길이 30: RNN 정확도 61.7% / 펼친 MLP 모양 오류 (1000x90 and 60x64)
길이 40: RNN 정확도 46.6% / 펼친 MLP 모양 오류 (1000x120 and 60x64)
```

펼친 MLP

길이가 바뀌면 실행조차 안 된다

RNN

어떤 길이든 받아 준다
하지만 길이 40에서 50% 근처: 받는 것과 잘하는 것은 다르다

교안과 차이

교안 86.6 / 57.5 / 48.2%
여기 87.9 / 61.7 / 46.6% (학습 결과라 조금씩 다름)

11.4 개념: 귓속말 놀이, 거듭 곱하기의 함정

```
h_100 = tanh(... h_99 W_h ...)
h_99  = tanh(... h_98 W_h ...)
...
∂손실/∂h_0 = ∂손실/∂h_100 × (W_h × tanh′) × (W_h × tanh′) × ... (100번)
```

한 번 곱하는 크기	10번	50번	100번	결과
0.5	0.001	8.9e-16	7.9e-31	소실
0.9	0.35	0.005	0.00003	소실
1.0	1	1	1	유지
1.1	2.6	117	13,800	폭주

귓속말 놀이

한 사람 거칠 때마다 말이 조금씩 줄어들면(또는 부풀면) 100명 뒤엔 사라진다(터진다)
층 대신 시간 방향으로 일어나는 기울기 소실·폭주

11.4 b09 거듭 곱하기와 tanh의 기울기

```
for 배율 in [0.5, 0.9, 1.0, 1.1, 1.5]:
    결과 = [배율 ** 횟수 for 횟수 in (10, 50, 100)]
    print(배율, 결과)

# 역전파 한 단계마다 tanh의 기울기(1 - tanh²)가 곱해진다
z = torch.tensor([0.0, 1.0, 2.0, 3.0], requires_grad=True)
torch.tanh(z).sum().backward()
print("tanh의 기울기:", z.grad)
```

```
0.5 거듭 곱하기 10번  0.000977 50번  8.88e-16 100번  7.89e-31
0.9 거듭 곱하기 10번     0.349 50번   0.00515 100번  2.66e-05
1.0 거듭 곱하기 10번         1 50번         1 100번         1
1.1 거듭 곱하기 10번      2.59 50번       117 100번  1.38e+04
1.5 거듭 곱하기 10번      57.7 50번  6.38e+08 100번  4.07e+17
tanh의 기울기: [1.0, 0.42, 0.0707, 0.0099]
```

tanh′ ≤ 1

z = 0일 때 최대 1, 값이 커지면 0에 가깝다
→ 곱할수록 더 작아지기 쉽다

1보다 조금만 달라도

0.9를 100번: 0.00003
1.1을 100번: 13,800

RNN에서는

시점 수만큼 곱해진다
길이 100 = 100번 곱하기

11.4 b10 시점별 기울기를 재 보기

```
x = torch.randn(1, 100, 8, requires_grad=True)    #길이 100

def 시점별_기울기(rnn):
    출력, _ = rnn(x)
    기울기, = torch.autograd.grad(출력[0, -1].sum(), x)
    return 기울기[0].norm(dim=1)                   #시점마다 크기

기본 = nn.RNN(8, 32, batch_first=True)
큰_가중치 = nn.RNN(8, 32, batch_first=True)        #같은 초기값
with torch.no_grad():
    큰_가중치.weight_hh_l0.mul_(4)                 # W_hh 4배
```

```
시점             99        95        90        80        50         0
기본 RNN    2.02e+00  1.85e-01  5.55e-03  9.52e-06  3.01e-14  0.00e+00
W_h 4배     1.54e+00  2.21e+00  1.55e+01  1.69e+01  2.63e+03  7.15e+07
```

읽는 법

마지막 출력이 t번째 입력에 얼마나 민감한가
99는 바로 앞, 0은 가장 먼 과거

기본 RNN

99 → 50: 2.02 → 3e-14
시점 0은 0 (float 범위 밑으로 사라짐)

W_h 4배

반대로 먼 과거일수록 폭발 (교안 1.67e+07, 여기 7.15e+07)

11.4 두 문제의 처방은 다르다

폭주 → 클리핑

```
nn.utils.clip_grad_norm_(
    모델.parameters(), max_norm=1.0)
```

한 걸음의 크기에 상한을 둔다 (8.9절)
순환 신경망 학습에서는 거의 기본으로 켠다

소실 → 구조를 바꾼다

작아진 기울기를 키울 방법은 없다
클리핑으로는 해결되지 않는다

답은 다음 차시의 LSTM과 GRU

Q. tanh 대신 ReLU면?

nonlinearity="relu"로 바꿀 수 있지만 상한이 없어 메모가 커지고 폭주가 쉬워진다
W_h를 거듭 곱하는 구조는 그대로라 소실도 그대로 (직접 실행 Q2에서 확인)

직접 실행: 2차시

복사본에서 고치고, 먼저 예상해서 적기

1 b07, b08을 실행하고 펼친 MLP의 오류 "(1000x72 and 60x64)"를 읽어 보자. 72와 60은?

2 b10 복사본에서 기본 RNN을 nonlinearity="relu"로 바꾸면 시점 50의 기울기는 tanh(3.01e−14)보다 큰가?

```
nn.RNN(8, 32, batch_first=True, nonlinearity="relu")
```

3 b09 복사본의 배율 목록에 0.99와 1.01을 더하면 100번 곱한 값은?

직접 실행 정답: 2차시

1

72 = 길이 24 × 원-핫 3 (실제 들어온 입력)
60 = 20 × 3 (Linear가 기대하는 입력). 칸 수가 안 맞아 곱셈 불가

2

아니다. 4.70e-19 (더 작다), 시점 0은 여전히 0
ReLU로 바꿔도 W_h를 거듭 곱하는 구조는 그대로

3

0.99 → 0.366, 1.01 → 2.70
1에서 1%만 벗어나도 100번이면 크게 달라진다

확인 문제: 2차시

1 RNN 학습 중 손실이 갑자기 nan이 됐다. 먼저 해 볼 일은?

가 메모 크기를 두 배로 나 clip_grad_norm_ 으로 상한
다 batch_first=False 로 라 순서열을 더 길게

2 짝맞나("(()))(")의 결과와, 몇 번째 글자에서 판정이 끝나는지?

3 개수만 보는 MLP가 학습 데이터에서도 50%인 이유는?

4 b10에서 기본 RNN의 시점 0 기울기가 0.00e+00인 이유는?

확인 문제 정답: 2차시

1
나
기울기 폭주 → 걸음 크기에 상한을 두는 클리핑

2
False
다섯 번째 ')'에서 깊이가 −1이 되어 바로 False

3
짝이 맞든 틀리든 '(' 10개, ')' 10개로 입력(개수)이 완전히 같다
같은 입력에 정답이 반반 → 찍기

4
float32로 표현할 수 있는 가장 작은 수(약 1e−45)보다 작아져 0이 됐다
그 시점은 학습에 아무 영향이 없다

2차시 정리

01

괄호 짝 문제에서 개수만 보는 모델은 50%. 펼친 MLP는 길이가 바뀌면 쓸 수 없고, RNN은 한 벌의 가중치로 어떤 길이든 받는다

02

받아 주는 것과 처음 보는 길이에서 잘하는 것은 다르다 (길이 40에서 50% 근처)

03

시점마다 기울기를 곱하니 사라지거나 터진다. 폭주는 클리핑으로, 소실은 구조로

다음: 기억을 따로 나르는 벨트, LSTM

3 LSTM, GRU와 긴 기억 시험

책 11.5 ~ 11.7 · b11 ~ b14

학습 목표

1. LSTM의 벨트와 문 셋을 손으로 계산하고 nn.LSTM과 맞춘다
2. 덧셈으로 고쳐 쓰는 벨트가 기울기를 살리는 이유를 말한다
3. RNN·GRU·LSTM의 비용과 긴 기억 성적을 숫자로 비교한다

책 범위

11.5 ~ 11.7

예제 파일

b11_lstm_cell · b12_params · b13_adding_data
b14_adding_train (19초)

11.5 개념: 기억을 따로 나르는 벨트

메모를 매번 새로 쓰지 말고, 지울 것만 지우고 더할 것만 더하자

벨트 c ── × 망각문 (덜어 내기) ── + 입력문 × 후보 (올려놓기) ──▶ 다음 시점

이름	하는 일	값의 범위
망각문 f	벨트 위의 기억을 얼마나 남길지	0~1 (시그모이드)
입력문 i	새 후보를 얼마나 올려놓을지	0~1 (시그모이드)
후보 g	올려놓을 새 내용	−1~1 (tanh)
출력문 o	벨트에서 지금 얼마나 꺼내 보일지	0~1 (시그모이드)

```
c_t = f * c_(t-1) + i * g
h_t = o * tanh(c_t)
```

문은 왜 시그모이드?

0이면 닫힘, 1이면 활짝 열림
0~1 값을 만드는 함수라서

LSTM은 상태가 둘: 벨트 c와 메모 h

11.5 b11 손으로 만드는 LSTM 셀

```
lstm = nn.LSTM(input_size=3, hidden_size=4, batch_first=True)

def lstm_셀(x, h, c):
    점수 = (x @ lstm.weight_ih_l0.T + lstm.bias_ih_l0
            + h @ lstm.weight_hh_l0.T + lstm.bias_hh_l0)   # 배치, 4 × 4)
    i, f, g, o = 점수.chunk(4, dim=1)     # PyTorch순서: 입력·망각·후보·출력
    입력문, 망각문, 출력문 = sigmoid(i), sigmoid(f), sigmoid(o)
    후보 = torch.tanh(g)
    c = 망각문 * c + 입력문 * 후보        # 남길 만큼 남기고 올릴 만큼 더한다
    h = 출력문 * torch.tanh(c)              #꺼내 보일 만큼만
    return h, c

출력, (h_n, c_n) = lstm(순서열)          # 상태가 두 개
```

```
weight_ih_l0: (16, 3) weight_hh_l0: (16, 4)
출력: (2, 5, 4) h_n: (1, 2, 4) c_n: (1, 2, 4)
h 차이: 2.9802322387695312e-08  c 차이: 3.725290298461914e-08
기억 c : [0.123, 0.341, -0.288, 0.479]
메모 h : [0.076, 0.158, -0.147, 0.244]
```

가중치가 4배

(16, 3) = 4 × 메모 4
문 셋 + 후보 하나를 한 번에 계산한 뒤 chunk(4)로 나눔

순서가 중요

i, f, g, o 순서를 틀리면 값이 달라진다 (직접 실행 Q2)

돌려주는 것

출력, (h_n, c_n)
RNN과 달리 괄호로 두 개를 묶어 준다

11.5 왜 기울기가 살아남을까

RNN: 새로 만든다

```
h_t = tanh(… h_(t-1) @ W_h …)
```

거꾸로 갈 때마다 W_h와 tanh의 기울기가 곱해진다 → 사라지거나 터진다

LSTM: 고쳐 쓴다

```
c_t = f * c_(t-1) + i * g
```

거꾸로 갈 때 한 단계마다 곱해지는 것은 사실상 f 하나
f가 1 근처면 기울기가 거의 그대로 흐른다

망각문을 여는 것도 배운다

"이건 기억해야 해"라고 판단하면 망각문을 1 가까이 연다. 그 값을 모델이 학습한다
덧셈으로 이어지는 길: ResNet의 지름길과 같은 아이디어

11.6 개념: 문을 두 개로 줄인 GRU

2014년. 벨트와 메모를 하나로 합치고, 문을 둘만 둔다

갱신문 z

지난 메모를 얼마나 남기고 새 후보로 얼마나 바꿀지
(망각문과 입력문을 하나로 묶었다)

리셋문 r

새 후보를 만들 때 지난 메모를 얼마나 참고할지

```
새 메모 = (1 - z) * 새 후보 + z * 지난 메모
```

남기는 양 + 바꾸는 양 = 1

뭔가를 남기면 그만큼 덜 바꾼다. 상태는 h 하나 (LSTM의 c가 없음)
LSTM과 GRU 중 무엇이 나은지는 문제마다 다르다 → 검증 데이터로 고른다

11.6 b12 파라미터 비용: 한 벌, 세 벌, 네 벌

```
입력_크기, 메모_크기 = 8, 32
for 종류 in [nn.RNN, nn.GRU, nn.LSTM]:
    층 = 종류(입력_크기, 메모_크기, batch_first=True)
    개수 = sum(p.numel() for p in층.parameters())
    출력, 상태 = 층(torch.randn(2, 10, 입력_크기))

기본 = 메모_크기 * 입력_크기 + 메모_크기 * 메모_크기 + 2 * 메모_크기
print(f"RNN 한 벌: {기본:,}개 → GRU 3벌, LSTM 4벌")
```

```
RNN  파라미터 1,344개 가중치 줄 수  32 상태 (1, 2, 32)
GRU  파라미터 4,032개 가중치 줄 수  96 상태 (1, 2, 32)
LSTM 파라미터 5,376개 가중치 줄 수 128 상태 [(1, 2, 32), (1, 2, 32)]
RNN 한 벌: 1,344개 → GRU는 3벌 4,032개, LSTM은 4벌 5,376개
```

한 벌 = RNN 하나

32×8 + 32×32 + 32 + 32 = 1,344

GRU

문 둘 + 후보 = RNN 계산 세 번
가중치 줄 수 96 = 3 × 32

LSTM

문 셋 + 후보 = 네 번
상태가 둘: _, (h_n, c_n) = 모델(x)

11.7 b13 긴 기억 시험: 멀리 떨어진 두 수 더하기

```
def 더하기_문제(개수, 길이, g):
    값 = torch.rand(개수, 길이, generator=g)       # 0~1 숫자 줄
    표시 = torch.zeros(개수, 길이)
    앞 = torch.randint(0, 길이 // 2, (개수,), generator=g)
    뒤 = torch.randint(길이 // 2, 길이, (개수,), generator=g)
    표시[torch.arange(개수), 앞] = 1.0             # 앞 절반에 하나
    표시[torch.arange(개수), 뒤] = 1.0             # 뒤 절반에 하나
    x = torch.stack([값, 표시], dim=-1)            # (개수, 길이, 2)
    y = (값 * 표시).sum(dim=1, keepdim=True)       #표시된 두 수의 합
    return x, y

_, y = 더하기_문제(100_000, 50, g)
print(f"늘 1.0이라고 답할 때 MSE: {((y - 1.0) ** 2).mean():.4f}")
```

```
숫자: [0.5, 0.77, 0.09, 0.13, 0.31, 0.63, 0.49, 0.9, 0.46, 0.63]
표시: [0, 1, 0, 0, 0, 0, 1, 0, 0, 0]
정답: 1.2583
늘 1.0이라고 답할 때 MSE: 0.1663
```

왜 좋은 시험인가

앞 절반의 표시된 숫자를 줄 끝까지 기억해야 풀린다
길이를 늘리면 기억할 거리가 늘어남

기준선 0.166

아무것도 못 배우고 평균(1.0)만 말할 때의 MSE
이 근처면 못 배운 것

0.1663의 정체

두 균등분포 합의 분산 = 1/12 + 1/12

11.7 b14 RNN, LSTM, GRU 비교

```
class 더하기_모델(nn.Module):
    def __init__(self, 종류):                          # nn.RNN / LSTM / GRU
        super().__init__()
        self.순환 = 종류(2, 32, batch_first=True)
        self.답 = nn.Linear(32, 1)
    def forward(self, x):
        출력, _ = self.순환(x)                         # LSTM의 (h, c)도 _로
        return self.답(출력[:, -1])

for 단계 in range(1, 1501):                             #매번 새 문제 64개
    ...
    nn.utils.clip_grad_norm_(모델.parameters(), max_norm=1.0)
```

```
길이 10 RNN  시험 MSE 0.0108 첫 시점 기울기 3.0e-01
   300번마다 학습 손실 [0.143, 0.062, 0.022, 0.014, 0.011]
길이 10 LSTM 시험 MSE 0.0002 첫 시점 기울기 3.5e-01
   300번마다 학습 손실 [0.088, 0.008, 0.001, 0.0, 0.0]
길이 10 GRU  시험 MSE 0.0002 첫 시점 기울기 3.4e-01
   300번마다 학습 손실 [0.008, 0.002, 0.001, 0.0, 0.0]
길이 50 RNN  시험 MSE 0.1582 첫 시점 기울기 2.5e-08
   300번마다 학습 손실 [0.164, 0.156, 0.182, 0.16, 0.174]
길이 50 LSTM 시험 MSE 0.0027 첫 시점 기울기 2.7e-01
   300번마다 학습 손실 [0.16, 0.154, 0.031, 0.005, 0.003]
길이 50 GRU  시험 MSE 0.0005 첫 시점 기울기 8.7e-02
   300번마다 학습 손실 [0.15, 0.004, 0.002, 0.001, 0.001]
```

길이 50의 RNN

0.158 ≈ 기준선 0.166 → 못 배움
학습 뒤에도 첫 시점 기울기 2.5e-08

평평하다가 갑자기

LSTM 손실 0.16 → 0.154 → 0.031
망각문을 여는 법을 배우는 데 시간이 걸린다
초반에 안 줄어든다고 일찍 포기하지 않기

GRU

가장 빨리 내려갔다

직접 실행: 3차시

1번을 먼저 걸어 두고 기다리는 동안 2·3번

1 b14 복사본에서 길이 목록을 [100]으로 바꿔 먼저 실행해 두자 (40초 안팎). 셋의 시험 MSE는?

```
for 길이 in [100]:
```

2 b11 복사본에서 i, f, g, o를 f, i, g, o로 바꾸면 h 차이와 c 차이는?

3 b12 복사본에서 메모_크기를 64로 바꾸면 파라미터 수는 몇 배?

직접 실행 정답: 3차시

1. RNN 0.1645 (기준선 수준, 첫 시점 기울기 0)
LSTM 0.0053 · GRU 0.0010
LSTM은 1,200번째 즈음에야 손실이 뚝 떨어졌다

2. h 차이 0.129, c 차이 0.220
0이 아니다 → 순서를 틀리면 다른 계산. PyTorch는 i, f, g, o 순서로 묶어 둔다

3. RNN 4,736 · GRU 14,208 · LSTM 18,944
약 3.5배 (2배가 아니다: W_h가 64×64로 제곱만큼 커져서)

확인 문제: 3차시

1 LSTM이 먼 과거까지 기울기를 전하는 핵심은?

가 파라미터가 RNN의 4배라서
나 벨트를 덧셈으로 고쳐 쓰고 망각문을 배워서
다 tanh 대신 시그모이드라서 라 클리핑이 들어 있어서

2 nn.LSTM(10, 20)과 nn.GRU(10, 20)의 파라미터 수는?

RNN(10, 20)은 640개

3 망각문 = 1, 입력문 = 0이면 벨트 c는? 망각문 = 0, 입력문 = 1이면?

```
c = f × c + i × g
```

4 길이 50 RNN의 학습 손실이 0.16 근처에서 안 내려간 뜻은?

확인 문제 정답: 3차시

1 나
곱셈 대신 덧셈으로 이어지는 벨트 + 망각문을 1
근처로 여는 법을 배움

2 LSTM 2,560개 (4벌)
GRU 1,920개 (3벌)

3 ① c가 그대로 유지
② 이전 기억을 다 지우고 후보로 바꿈

4 늘 1.0이라고 답하는 것(0.166)과 같은 수준
앞쪽 숫자를 기억하지 못해 평균만 말하고 있다

3차시 정리

01

LSTM: 벨트 c와 문 셋(망각·입력·출력). 덧셈으로 고쳐 쓰는 벨트 덕분에 기울기가 먼 과거까지 흐른다. 상태는 (h_n, c_n)

02

GRU: 문 둘(갱신·리셋), 파라미터는 RNN의 3배 (LSTM은 4배). 무엇이 나은지는 검증 데이터로

03

길이 50 더하기: RNN 0.158(못 배움), LSTM 0.0027, GRU 0.0005. 곡선은 평평하다가 갑자기 내려간다

다음: 층 쌓기·양방향, 길이가 다른 순서열, 실제 시계열 예측

4

층 쌓기, 꾸러미, 이산화탄소 예측

책 11.8 ~ 11.10 · b15 ~ b22

학습 목표

1. 층 수·방향 수에 따른 출력과 h_n의 모양을 읽는다
2. pack_padded_sequence로 채운 칸을 건너뛰게 한다
3. 기준선부터 세우고 시계열을 예측·이어 예측한다

책 범위

11.8 ~ 11.10

예제 파일

b15_stack_bi · b16_pad_problem · b17_pack · b18_paren_pack (13초)
b19_co2_look · b20_co2_baseline · b21_co2_rnn (11초) · b22_co2_forecast

11.8 b15 두 층, 그리고 거꾸로 읽기

```
x = torch.randn(2, 7, 8)                          # 배치 2, 길이 7, 특성 8)
설정들 = {"한 층": {}, "두 층": dict(num_layers=2, dropout=0.2),
          "양방향": dict(bidirectional=True),
          "두 층 양방향": dict(num_layers=2, bidirectional=True)}
for 이름, 설정 in 설정들.items():
    lstm = nn.LSTM(8, 16, batch_first=True, *설정)
    출력, (h_n, c_n) = lstm(x)

# 양방향 출력: 앞 16칸은 정방향, 뒤 16칸은 역방향
print(torch.allclose(h_n[0],출력[:, -1, :16]))  # 정방향은 끝에서 끝남
print(torch.allclose(h_n[1],출력[:, 0, 16:]))   # 역방향은 처음에서 끝남
양쪽_요약 = torch.cat([h_n[-2], h_n[-1]], dim=1)  #분류엔 둘을 이어 붙임
```

```
한 층: 출력 (2, 7, 16), h_n (1, 2, 16), 파라미터 1,664개
두 층: 출력 (2, 7, 16), h_n (2, 2, 16), 파라미터 3,840개
양방향: 출력 (2, 7, 32), h_n (2, 2, 16), 파라미터 3,328개
두 층 양방향: 출력 (2, 7, 32), h_n (4, 2, 16), 파라미터 9,728개
정방향 마지막 = 출력[:, -1, :16]? True
역방향 마지막 = 출력[:, 0, 16:]? True
양쪽 요약: (2, 32)
```

h_n의 첫 자리

층 수 × 방향 수
양방향 출력 = 정방향 16칸 + 역방향 16칸

역방향의 '마지막'

맨 처음 시점. 거꾸로 읽었으니까
그래서 h_n[1] = 출력[:, 0, 16:]

양방향을 쓰면 안 되는 곳

다음 달 예측, 다음 글자 생성: 미래 정보 누수
문장 분류처럼 다 보고 판단할 때만

11.9 b16 패딩이 메모를 오염시킨다

```
rnn = nn.RNN(2, 4, batch_first=True)
짧은 = torch.randn(3, 2)                              #길이 3
긴 = torch.randn(6, 2)                                #길이 6
배치 = pad_sequence([짧은, 긴], batch_first=True)   # 짧은 쪽 뒤를 0으로 → (2, 6, 2)

출력, h_n = rnn(배치)
_, h_혼자 = rnn(짧은.unsqueeze(0))                   #짧은 것만 따로
print("혼자 읽은 마지막 메모:", h_혼자[0, 0])
print("배치의 h_n          :", h_n[0, 0])
print("배치의 3번째 시점   :", 출력[0, 2])
```

```
패딩한 배치: (2, 6, 2)
혼자 읽은 마지막 메모: [0.172, -0.593, 0.522, 0.458]
배치의 h_n          : [-0.349, -0.04, 0.226, -0.086]
배치의 3번째 시점   : [0.172, -0.593, 0.522, 0.458]
```

pad_sequence

길이가 다른 문장을 한 텐서로 묶으려고
짧은 쪽 뒤를 0으로 채운다

문제

진짜 3글자를 읽은 뒤 채운 0을 세 번 더
읽으며 메모를 계속 고쳐 썼다
회의가 끝났는데 녹음기가 계속 돈 셈

진짜 답은

3번째 시점의 출력 = 혼자 읽은 메모

11.9 b17 길이를 알려 주고 꾸러미로 싸기

```
문장들 = [torch.randn(3, 2), torch.randn(6, 2), torch.randn(4, 2)]
길이 = torch.tensor([len(s) for s in문장들])
배치 = pad_sequence(문장들, batch_first=True)

꾸러미 = pack_padded_sequence(배치, 길이, batch_first=True,
                              enforce_sorted=False)   #길이 순 정렬 안 해도 됨
print(꾸러미.data.shape, 꾸러미.batch_sizes)

출력_꾸러미, h_n = rnn(꾸러미)                        # RNN에 꾸러미를 그대로
# h_n[0, i] == 문장 i를 혼자 읽은 마지막 메모

출력, 출력_길이 = pad_packed_sequence(출력_꾸러미, batch_first=True)
```

```
실제 계산할 칸: (13, 2) 시점별 문장 수: [3, 3, 3, 2, 1, 1]
문장 0 (길이 3): 혼자 읽은 것과 같은가? True
문장 1 (길이 6): 혼자 읽은 것과 같은가? True
문장 2 (길이 4): 혼자 읽은 것과 같은가? True
되돌린 출력: (3, 6, 4) [3, 6, 4]
채운 칸의 출력은 0: 0.0
```

꾸러미

진짜 칸 3 + 6 + 4 = 13만 계산
batch_sizes: 시점마다 아직 안 끝난 문장 수

결과

RNN은 문장마다 자기 길이까지만 읽는다 = 혼자 읽은 것과 같다

되돌리기

pad_packed_sequence로 다시 (배치, 길이, 메모)
채운 칸의 출력은 0

11.9 b18 여러 길이로 학습하고 더 긴 괄호로 시험

```
학습_길이들 = list(range(2, 21, 2))                # 2, 4, ..., 20
괄호_학습, y_학습 = 괄호_만들기(6000, 학습_길이들, 씨앗=0)
X_학습, 길이_학습 = 원핫으로(괄호_학습)          # (6000, 20, 3) +길이

class 괄호_판정기(nn.Module):
    def forward(self, x, 길이):
        if self.꾸릴까:
            x = pack_padded_sequence(x,길이, batch_first=True,
                                     enforce_sorted=False)
        _, 상태 = self.순환(x)
        h_n = 상태[0] if isinstance(상태, tuple) else 상태    # LSTM은 (h, c)
        return self.판단(h_n[-1])
# 15에폭, 처음 보는 길이 30, 40, 60으로 시험
```

```
학습 배치 모양: (6000, 20, 3) 가장 짧은/긴 길이: 2 20
모델  꾸러미  검증(2~20)  길이 30  길이 40  길이 60
RNN   안 씀     99.7%    90.7%    85.4%    77.5%
RNN   씀       100.0%    94.2%    81.2%    67.2%
LSTM  안 씀     100.0%    86.1%    72.3%    53.9%
LSTM  씀       100.0%    98.2%    89.6%    70.0%
```

꾸러미 + LSTM

길이 30·40에서 가장 좋다 (98.2%, 89.6%: 교안과 같음)

길이 20만 본 RNN과 비교

2차시 b08 길이 30: 61.7%
→ 길이를 섞고 싸면 94.2%

교안과 차이

꾸러미 없는 RNN은 교안 78.3%, 여기 99.7%
학습이 흔들리는 설정이라 실행마다 크게 달라진다

11.10 b19 마우나로아 이산화탄소 둘러보기

```
from statsmodels.datasets import co2

주간 = co2.load_pandas().data["co2"]     #주간 측정값 (ppm)
월간 = 주간.resample("MS").mean()         #달마다 평균 (MS: 매달 1일)
월간 = 월간.interpolate()                 #빈 달은 앞뒤 사이 값으로
연평균 = 월간.groupby(월간.index.year).mean()
print(연평균[[1960, 1970, 1980, 1990, 2000]])
print("2000년 달마다:", 월간["2000"].tolist())
```

```
(2284,) 빈 값: 59
달 수: 526 빈 달: 5
1958-03-01 ~ 2001-12-01
연평균: {1960: 316.9, 1970: 325.7, 1980: 338.6, 1990: 354.1, 2000: 369.4}
2000년 달마다: [369.0, 369.4, 370.4, 371.5, 371.6, 371.6, 369.9, 368.0, 366.5, 366.7,
368.1, 369.4]
```

resample("MS")

주간 2,284개 → 월간 526개월

interpolate()

빈 달 5개를 앞뒤 값 사이로 메움

두 가지 패턴

추세: 316.9 (1960) → 369.4 (2000)
계절성: 5~6월 371.6 / 9월 366.5
좋은 모델은 둘 다 잡아야 한다

11.10 b20 기준선부터 세우기

```
창 = 24                                    #지난 24개월로 다음 달
X = 값.unfold(0, 창, 1)[:-1]              # (502, 24)
y = 값[창:]
# 시간 순서로 자르기: ~1990 학습, 1991~1995 검증, 1996~ 테스트
학습 = torch.tensor(날짜 < "1991-01-01")
...
지난달 = X[:, -1]                                  #기준선 1
작년_같은달 = X[:, -12] + (X[:, -1] - X[:, -13])   #기준선 2
```

```
X (502, 24) 학습 370 검증 60 테스트 72
지난달 그대로: 검증 RMSE 1.305, 테스트 RMSE 1.253 ppm
작년 같은 달 + 1년 증가분: 검증 RMSE 0.340, 테스트 RMSE 0.428 ppm
```

시간 순서로 나눈다

무작위로 섞으면 미래로 과거를 맞히는 셈
(누수)

기준선 2

작년 같은 달 + 지난 1년 동안 오른 양
계절과 추세를 둘 다 반영

테스트 0.428 ppm

사람이 만든 규칙의 성적
이걸 못 이기면 신경망을 쓸 이유가 없다

11.10 b21 RNN, LSTM, GRU로 예측하기

```
# 절댓값 대신 '마지막 달과의 차이'로 → '다음 달 변화량' 맞히기
마지막 = X[:, -1:]
입력, 목표 = X - 마지막, y - 마지막[:, 0]
척도 = 입력[학습].std()                    #학습 구간에서만 잰 크기
입력, 목표 = 입력 / 척도, 목표 / 척도

class 예측기(nn.Module):
    def forward(self, x):
        출력, _ = self.순환(x.unsqueeze(-1))    # (배치, 24) → (배치, 24, 1)
        return self.답(출력[:, -1]).squeeze(-1)

for 에폭 in range(1, 201):
    ...
    if 검증_오차 < 최고[0]:                #검증이 가장 좋았던 순간 보관
        최고 = (검증_오차, 에폭, copy.deepcopy(모델.state_dict()))
```

```
RNN : 137에폭 선택, 검증 RMSE 0.276, 테스트 RMSE 0.325 ppm
LSTM:  47에폭 선택, 검증 RMSE 0.315, 테스트 RMSE 0.358 ppm
GRU : 148에폭 선택, 검증 RMSE 0.286, 테스트 RMSE 0.321 ppm
저장한 모델: RNN
```

왜 차이로 바꾸나

CO2 값은 계속 커져 학습 때 못 본 범위가 테스트에 나온다
변화량은 범위가 비슷하다

척도는 학습 구간에서만

검증·테스트 정보가 섞이지 않게

결과

셋 다 기준선 2(검증 0.340, 테스트 0.428)보다 낮다
검증 1등은 평범한 RNN: 창이 24개월이라 긴 기억이 필요 없다

11.10 b22 여러 달을 이어서 내다보기

```
저장본 = torch.load("co2_모델.pt", weights_only=True)
모델 = 예측기(getattr(nn, 저장본["종류"]))      # "RNN" → nn.RNN
...
# 1995년 12월까지만 알고, 자기 예측을 이어 붙이며 72개월
with torch.no_grad():
    for _ in range(개월):
        창 = 모델_예측[-24:]
        변화 = 모델((창 - 창[-1]).unsqueeze(0) / 척도) * 척도
        모델_예측 = torch.cat([모델_예측, 창[-1:] + 변화])
        다음 = 기준선[-12] + (기준선[-1] - 기준선[-13])
        기준선 = torch.cat([기준선, 다음.unsqueeze(0)])
```

```
개월 72
RNN 이어 예측: 첫 12개월 RMSE 0.353, 72개월 RMSE 1.934, 2001년 12월 오차 -2.86 ppm
기준선 이어 예측: 첫 12개월 RMSE 0.372, 72개월 RMSE 0.656, 2001년 12월 오차 -0.48 ppm
연간 증가량: 1959~1990년 평균 1.23ppm, 1995~2001년 평균 1.67ppm
```

이어 예측

예측값을 다시 입력으로 넣어 다음 달을 예측
작은 오차가 72번 쌓인다

첫 12개월 vs 72개월

12개월은 RNN이 낫지만(0.353), 72개월은 기준선이 훨씬 낫다(0.656 vs 1.934)

교훈

한 달 뒤를 잘 맞히는 모델이 6년 뒤도 잘 맞히는 것은 아니다

직접 실행: 4차시

1번을 먼저 걸어 두고 기다리는 동안 2·3번

1 b21 복사본에서 창 = 24를 12로, 저장 이름을 "co2_창12.pt"로 바꿔 먼저 실행. 검증 1등과 테스트 RMSE는?

저장 이름을 꼭 바꾼다 (b22가 co2_모델.pt를 읽는다)

2 b15 복사본에 "세 층 양방향": dict(num_layers=3, bidirectional=True)를 더하면 h_n 모양은? 먼저 예상

3 b17 복사본의 문장들에 torch.randn(5, 2)를 더하면 계산할 칸과 시점별 문장 수는?

길이 [3, 6, 4, 5]

직접 실행 정답: 4차시

1. 검증 1등은 여전히 RNN: 검증 0.272, 테스트 0.328 ppm
(LSTM 0.306/0.360, GRU 0.289/0.346) 창 24일 때와 거의 같다

2. h_n (6, 2, 16) = 층 3 × 방향 2
출력은 그대로 (2, 7, 32), 파라미터 16,128개

3. (18, 2): 3 + 6 + 4 + 5 = 18칸
batch_sizes [4, 4, 4, 3, 2, 1]

확인 문제: 4차시

1 nn.LSTM(8, 16, num_layers=2, bidirectional=True)의 h_n 첫 자리는?

가 1　나 2　다 4　라 32

2 길이 [2, 5, 3]인 세 문장을 pack하면 data의 행 수와 batch_sizes는?

3 다음 달 CO2를 예측하는 모델에 양방향 LSTM을 쓰면 안 되는 이유는?

4 작년 같은 달 = 360, 지난달 = 365, 13달 전 = 363이면 기준선 2의 예측은?

```
X[-12] + (X[-1] - X[-13])
```

확인 문제 정답: 4차시

1

다 4
층 수 2 × 방향 수 2

2

data 10행 (2 + 5 + 3)
batch_sizes [3, 3, 2, 1, 1]

3

역방향은 미래(뒤 시점)를 읽는다
예측할 땐 미래를 모르니 쓸 수 없다 (누수)

4

362
360 + (365 − 363)

4차시 정리

01

h_n의 첫 자리는 층 수 × 방향 수. 양방향은 미래를 보므로 예측·생성에는 쓰지 않는다

02

패딩만 하면 채운 칸까지 읽어 메모가 오염된다. pack_padded_sequence(…, enforce_sorted=False)

03

시계열: 기준선부터, 상대값으로 넣고, 검증으로 고른다. 이어 예측하면 오차가 쌓인다

다음: 속담을 읽고 새 속담을 짓는 글자 단위 LSTM

5

실습: 속담 짓는 글자 단위 LSTM

책 11.11 ~ 11.12 · b23 ~ b26

학습 목표

1. 입력을 한 칸 밀어 다음 글자 맞히기 문제를 만든다
2. nn.Embedding이 원-핫 @ 표와 같음을 확인한다
3. 확률대로 뽑아 글을 짓고, 결과를 정직하게 평가한다

책 범위

11.11 ~ 11.12

예제 파일

b23_proverb_file · b24_proverb_prep
b25_proverb_train (27초) · b26_proverb_generate

11.11 1단계 b23: 속담 117개를 파일로

```
# 저작권이 없는 옛 속담 117개. " / "로 나눠 한 줄에 몇 개씩
속담 = """
가는 말이 고와야 오는 말이 곱다 / 가는 날이 장날
가랑비에 옷 젖는 줄 모른다 / 가재는 게 편
…
"""
속담들 = [s.strip() for 줄 in 속담.strip().splitlines()
                    for s in 줄.split(" / ")]
경로 = Path("data/속담.txt")
경로.write_text("\n".join(속담들) + "\n", encoding="utf-8")
```

```
속담 117 개, 글자 1642 개 (줄바꿈 포함)
가장 짧은 속담: 눈엣가시
가장 긴 속담: 호랑이는 죽어서 가죽을 남기고 사람은 죽어서 이름을 남긴다
```

한 줄에 하나

" / "로 쪼개 한 줄에 속담 하나씩 저장
줄바꿈이 속담의 끝 표시

글자 1,642개

아주 작은 데이터
이 크기가 결과에 어떤 영향을 주는지
마지막에 본다

11.11 2단계 b24: 번호로 바꾸고 한 칸 밀기

```
글 = Path("data/속담.txt").read_text(encoding="utf-8")
글자들 = sorted(set(글))                          # 줄바꿈 "\n"도 한 글자
번호 = {글자: i for i, 글자 in enumerate(글자들)}
전체 = torch.tensor([번호[글자] for 글자 in 글])

# 입력을 한 칸 밀면 정답이 된다
조각 = 전체[:9]
입력, 정답 = 조각[:-1], 조각[1:]
```

```
글자 종류: 346 앞쪽 다섯: ['\n', ' ', '가', '각', '간']
번호로 바꾼 글: (1642,) [2, 65, 1, 135, 261, 1, 19, 242]
입력: 가는 말이 고와
정답: 는 말이 고와야
임베딩 표: (346, 8) → 꺼낸 벡터: (8, 8)
원-핫 @ 표와 같은가? True
```

한 칸 밀기

"가는 말이 고와" → "는 말이 고와야"
모든 시점이 다음 글자 맞히기 문제

정답을 매길 필요가 없다

글 자체가 문제와 정답이 된다
4일차의 거대 언어 모델도 이렇게 학습한다

11.11 nn.Embedding: 번호로 줄을 꺼내는 표

```
임베딩 = nn.Embedding(len(글자들), 8)        # (346, 8) 표
벡터 = 임베딩(입력)                          # 번호 8개 → (8, 8)
원핫 = nn.functional.one_hot(입력, len(글자들)).float()
torch.allclose(벡터, 원핫 @ 임베딩.weight)   # True
```

원-핫

글자 하나 = 숫자 346개, 거의 전부 0
표와 곱하면 한 줄만 남는다

임베딩

곱셈 없이 그 줄을 바로 꺼낸다
표의 숫자는 학습하며 함께 고쳐진다

결과가 같다

원-핫 @ 표 = 표에서 번호 줄 꺼내기 → 값은 같고 계산은 훨씬 싸다

11.11 3단계 b25: 임베딩 → LSTM → 다음 글자 점수

```
class 속담_LSTM(nn.Module):
    def __init__(self, 글자_수, 임베딩_크기=32, 메모_크기=128):
        super().__init__()
        self.임베딩 = nn.Embedding(글자_수, 임베딩_크기)
        self.lstm = nn.LSTM(임베딩_크기, 메모_크기, batch_first=True)
        self.다음_글자 = nn.Linear(메모_크기, 글자_수)
    def forward(self, x, 상태=None):
        출력, 상태 = self.lstm(self.임베딩(x), 상태)
        return self.다음_글자(출력), 상태      # 모든 시점의 다음 글자 점수

조각 = 전체[묶음.unsqueeze(1) + torch.arange(길이 + 1)]    # (64, 41)
점수, _ = 모델(조각[:, :-1])                                # (64, 40, 346)
손실 = cross_entropy(점수.reshape(-1, 346), 조각[:, 1:].reshape(-1))
```

```
파라미터: 138,650개 시작점 1602
 1에폭 손실 4.649 | 가을둡싼기 만 굳다글 | 호랑이한서
 5에폭 손실 2.467 | 가을 배기 있다 | 호랑이다글
10에폭 손실 0.893 | 가면 배가 있다 | 호랑이다
20에폭 손실 0.201 | 가면 배가 있다 | 호랑이다
40에폭 손실 0.105 | 가는 데 실 간다 | 호랑이는 죽어서 가죽을 남기고 사람은 죽어서 이름을 남긴다
60에폭 손실 0.091 | 가는 길 먹기 | 호랑이 굴에 가야 호랑이 새끼를 잡는다
```

조각 자르기

41글자를 아무 곳에서나 잘라 앞 40은 입력, 한 칸 민 40은 정답

reshape(−1, 346)

펴서 글자 하나 맞히기 2,560개(64 × 40)로 계산

손실이 줄면서

처음 손실 ≈ ln 346 ≈ 5.85 (찍기)
아무 글자 → 띄어쓰기·"~다" 말투 → 진짜 속담

11.11 4단계 b26: 새 속담 지어 보기

```
def 짓기(시작, g, 최대=40):
    x, 상태, 결과 = torch.tensor([[번호[c] for c in 시작]]), None, 시작
    with torch.no_grad():
        for _ in range(최대):
            점수, 상태 = 모델(x, 상태)            # 상태를 넘겨 이어 읽기
            다음 = torch.multinomial(점수[0, -1].softmax(dim=0), 1,
                                     generator=g).item()   #확률대로 뽑기
            if 글자들[다음] == "\n":
                break                                #줄바꿈 = 속담 끝
            결과 += 글자들[다음]
            x = torch.tensor([[다음]])           # 방금 뽑은 글자 하나만
    return 결과.strip()

지은_것 = [짓기("\n", g) for _ in range(30)]    #줄바꿈 다음 = 첫 글자부터
```

```
30개 중 원래 있던 속담 24개, 새로 지은 것 6개
  - 잘되면 제 탓 못되면 조상 리 간다
  - 호랑이도 제짝 말이 천 리 간다
  - 구는 도끼에 발등 찍힌다
  - 금은 사람은 생각도 않는데 김칫국부터 마신다
  - 땅 줄 고추가 더 맵다
  - 기는 등 건져 없는 놈 건져 놓으니 보따리 내놓으라 한다
시작 글자를 정해 주면:
  - 원숭이도 나무에서 떨어진다 / 원숭이도 나무에서 떨어진다 / 원숭이도 나무에서 떨어진다
  - 고양이 부뚜막에 먼저 올라간다 / 고양이도 밟아야 오는 말이 천 리 간다 / 고양이 목에 방울 달기
  - 티끌 모아 태산 / 티끌 모아 태산 / 티끌 모아 태산
  - 사람은 죽어서 이름을 남긴다 / 사람 속은 모른다 / 사람 속은 모른다
```

상태 이어 받기

앞 글자를 다시 읽지 않고 (h, c)만 넘겨 한 글자씩 (1차시 b05 이어 읽기)

multinomial

확률대로 뽑기 → 매번 다른 속담

갈아타기

두 속담에 같은 조각("제", "말", "천 리")이 나오는 자리에서 다른 속담으로 넘어간다

11.11 정직하게 평가하기, 11.12 RNN의 한계

평가

이해해서 짓는 것이 아니라 외운 것을 이어 붙이는 수준
글자 1,642개, 손실 0.091 = 거의 외웠다는 뜻

한계 1: 메모 하나

문장 전체를 크기가 정해진 메모 하나에 눌러 담는다
길수록 앞부분이 흐려진다

한계 2: 한 시점씩

100번째 글자를 읽으려면 99번째까지 다 읽어야 한다
GPU의 "한꺼번에 계산"을 못 쓴다

답: 모든 자리를 한꺼번에 돌아보는 어텐션과 트랜스포머 (2일차)

직접 실행: 5차시

b25는 30초 가까이 걸린다

1. b23 → b25 → b26 순서로 실행. 30개 중 원래 속담과 새것은 몇 개?

2. b26 복사본에서 multinomial 줄을 점수[0, -1].argmax().item()으로 바꾸면 30개는?

3. b26 복사본의 시작 글자 목록을 ["바늘", "말", "개"]로 바꾸면? 어느 속담끼리 이어 붙었나?

직접 실행 정답: 5차시

1

원래 있던 속담 24개, 새로 지은 것 6개
예) "땅 줄 고추가 더 맵다", "호랑이도 제짝 말이 천 리 간다"

2

30개 모두 "호랑이도 제 말 하면 온다" 하나
가장 확률 높은 글자만 고르니 같은 시작에선 늘 같은 문장

3

바늘 → "바늘 가는 데 실 간다"(원래 속담), "바늘 도 심은 데 팥 난다"(…심은 데 팥 난다와 이어 붙음)
말 → "말이 천 리 간다"(발 없는 말이…의 뒷부분), "말이라도 꿰어야 보배"(구슬이 서 말이라도…)
개 → "개구리 올챙이 적 생각 못 한다", "개 지붕 없다"
(뽑기 결과라 실행마다 다를 수 있음)

확인 문제: 5차시

1 글자 단위 모델의 학습 정답은 어떻게 만들었나?

가 사람이 좋다·나쁘다를 매겼다 나 입력을 한 칸 밀었다
다 원-핫을 정답으로 썼다 라 마지막 글자 하나만

2 조각이 "티끌 모아 태산"일 때 입력과 정답은?

3 nn.Embedding(346, 8)의 파라미터 수는?

4 b25 모델의 파라미터 수를 부분별로 계산하면?

임베딩 346×32 · LSTM(32, 128) · Linear(128, 346)

확인 문제 정답: 5차시

1 나
입력 = 조각[:−1], 정답 = 조각[1:]

2 입력 "티끌 모아 태"
정답 "끌 모아 태산"

3 2,768개 (346 × 8)
글자마다 8칸짜리 줄 하나

4 11,072 + 82,944 + 44,634 = 138,650개
LSTM = 4 × (128×32 + 128×128 + 256)

5차시 정리

01

입력을 한 칸 밀면 정답이 된다. 모든 시점이 다음 글자 맞히기 문제

02

nn.Embedding은 원-핫 @ 표와 같은 값을 곱셈 없이 꺼낸다

03

확률대로 뽑아 이어 붙인다. 작은 데이터에서는 외운 것을 이어 붙이는 수준에 머문다

1일차에 배운 것

1차시	(배치, 길이, 특성), 같은 가중치로 메모를 고쳐 쓰는 RNN
2차시	순서가 필요한 문제와 기울기 소실·폭주
3차시	덧셈으로 고쳐 쓰는 LSTM의 벨트, GRU, 긴 기억 시험
4차시	층·방향, 꾸러미, 기준선부터 세우는 시계열 예측
5차시	다음 글자 맞히기로 학습하고 확률대로 뽑아 짓기

내일 2일차 (6~10차시)

어텐션, d의 제곱근
셀프 어텐션과 마스크
멀티헤드, 위치 정보, 블록 조립
작은 GPT로 글 만들기
부품 빼 보기 실습